

Git Workshop Part 2

Tech GC

May 2026

1. Mental Model Review: Commits, Branches, and References
2. Merge Strategies and Team Integration
3. Rebase for Clean Feature Branches
4. Interactive Rebase: Editing Local History
5. Cherry-Pick: Moving Selected Commits
6. Git Reflog: Recovery and Repair
7. Stash, Worktrees, and Partial Staging
8. Lazygit: Visual Terminal Workflow
9. Remote Collaboration Policies
10. Fork and Pull Request Workflow
11. GitHub Actions: Automation Basics
12. Capstone Exercise: Clean Up a Messy Team Repository

Section 1

1. Mental Model Review: Commits, Branches, and References

Recap from Part 1

- **Commits:** Snapshots of repository state with hash id and parent links.
- **Branches:** Lightweight, movable pointers to specific commits.
- **HEAD:** Your current location indicator (where you are writing next).
- **Remote-tracking Branches:** Local references that mirror remote branch tips after fetch/pull (e.g., `origin/main`).

Core Key Insight

Branches do not contain commits. Branches point to commits, and commits point backward to their parents.

Visualizing the Graph

```
git log --graph --oneline --decorate --all
```

What this shows you:

- Exact history topology and branch divergence.
- Current position of `HEAD` relative to local and remote tracking branches.

Detached HEAD State

The Concept: Checking out a specific commit instead of a branch detaches HEAD from any branch name.

```
git checkout abc1234  
# HEAD now points directly to a commit
```

Warning: *New commits created here can become unreachable from normal branch names once you checkout another branch.*

Quick Recovery Options

The safest move is to give the detached state a name *before* you leave it:

```
# Save state immediately by creating a branch from where you are  
git checkout -b new-branch-name
```

Tip: We will come back to deeper recovery (e.g., when you have already switched away) in Section 6 “Git Reflog”.

A Tiny Example

```
git checkout abc1234          # HEAD detaches  
# ... experimental commit X ...  
git checkout -b try-x         # X is now safely on branch "try-x"
```

Understanding References

Reference Types

Type	Example	Behavior
Branch	<code>main</code> , <code>feature/login</code>	Moves forward dynamically with new commits
Tag	<code>v1.0.0</code> , <code>release-2.0</code>	Static snapshot marker; never moves
Remote-tracking	<code>origin/main</code>	Local mirror of remote state, updated by fetch/pull
Special	<code>HEAD</code> , <code>MERGE_HEAD</code>	Internal operational pointers

Useful Reference Shortcuts

- `HEAD~1` or `HEAD^` : Direct parent of current commit
- `HEAD~2` : Grandparent of current commit
- `main~3` : Three commits back from the tip of the main branch
- `origin/main@{yesterday}` : State of the tracking branch 24 hours ago

Section 2

2. Merge Strategies and Team Integration

How git merge Actually Works

Before talking about *strategies*, it helps to understand the **underlying algorithm**.

Almost every `git merge` is a **three-way merge**: Git finds the *merge base* (latest common ancestor of the two branches) and combines changes from both sides relative to that base.

```
A---B---C   (main)
  \
   D---E     (feature)
```

Merge base: A

"Left side" changes : A -> C

"Right side" changes : A -> E

Three-way merge combines both onto the result.

Key idea: Whether the result is a fast-forward, a no-ff merge commit, or a squashed single commit is a question of **how the result is recorded**, not which algorithm is used.

Integration Choices Overview

Fast-Forward Merge (--ff)

When the target branch has not moved since the source branch started, no actual three-way combination is needed. Git simply slides the branch pointer forward.

```
Before: A---B---C (main)
          \
            D---E (feature)
After:  A---B---C---D---E (main, feature)
```

No-Fast-Forward Merge (`--no-ff`)

Forces a real merge commit even when fast-forward is possible. The feature branch stays visible as a “bubble” in the graph.

```
Before: A---B---C (main)
          \
           D---E (feature)
After:  A---B---C-----M (main)
          \      /
           D---E (feature)
```

Squash Merge (--squash)

Condenses all incoming changes into a single brand-new commit on the target branch. The feature branch history is *not* recorded on `main`.

Before: A---B---C (main)

\

D---E---F (feature)

After: A---B---C---G (main) # G = D+E+F squashed

Decision Strategy Matrix

Strategy	Best Use Case	History Style	Extra Commit?
Fast-Forward (--ff)	Short-lived linear branches	Seamless line	No
No-FF (--no-ff)	Strict Feature/PR tracking	Grouped graph (bubble)	Yes (merge commit)
Squash (--squash)	Noisy local feature commits	Compact line	Yes (squashed commit)
Rebase + Merge	Want clean linear history but main moved	Seamless line	Optional

*All four are powered by the same three-way merge engine — they only differ in **what kind of commit (if any) is produced.***

Team Integration Workflows

Approach A: Linear History Preference

Keep the main line strictly straightforward. Clean chronological reading.

```
git checkout feature-branch
git rebase main
git checkout main
git merge feature-branch  # Results in a clean Fast-Forward
```

Approach B: Preserved Topology Preference

Explicitly show where work started, evolved, and was integrated.

```
git checkout main
git merge --no-ff feature-branch  # Guarantees a merge node
```

Worked Example

Suppose `main` has commits A-B-C and your `feature` branch has D-E branched from B.

```
# Snapshot 1: fast-forward not possible (main has C after B)
$ git checkout main
$ git merge feature          # creates merge commit M
# Result: A-B-C-M, with feature D-E pointing into M

# Snapshot 2: with --squash
$ git merge --squash feature
$ git commit -m "feat: add feature X"
# Result: A-B-C-G (single squashed commit, feature branch unchanged)

# Snapshot 3: with rebase first, then merge
$ git checkout feature && git rebase main    # D,E -> D',E' on top of C
$ git checkout main && git merge feature      # fast-forward to E'
# Result: A-B-C-D'-E' (clean linear)
```

Discussion Questions

- Which result is easiest to revert if the feature breaks production?
- Which result is easiest for a *new* reader of the project history?
- Which result hides the fact that a feature branch ever existed?

Section 3

3. Rebase for Clean Feature Branches

The Mechanics of Rebase

Rebase updates the starting parent commit of your branch, replaying your local changes sequentially on top of a new base commit.

Before Rebase:

```
A---B---C (main)
      \
       D---E (feature)
```

After 'git rebase main':

```
A---B---C (main)
      \
       D'---E' (feature)
```

Note: *D' and E' contain identical code changes to D and E, but carry distinct hash signatures and unique parent identifiers.*

Rebase Execution Steps

Step 1: Update your local tracking coordinates
`git checkout main && git pull`

Step 2: Begin replay process
`git checkout feature/my-feature`
`git rebase main`

Step 3: Handle individual files if conflicts occur, then proceed
`git add conflict_resolved.txt`
`git rebase --continue`

Rebase Interruption Controls

- `git rebase --continue` : Continue replaying commits after a conflict resolution.
- `git rebase --skip` : Drop the current conflicting commit entirely.
- `git rebase --abort` : Instantly halt and return your branch to pre-rebase status.

Why One Rebase Can Stop Multiple Times

Each replayed commit is applied **on top of the new base** in order. If commit `D` touches the same lines as recent `main` changes, you resolve once. If commit `E` *also* touches those lines, you resolve again. Each resolution is per-commit, not per-rebase.

```
Rebase plan: D -> E onto C
[step 1/2] applying D ...      CONFLICT in src/api.ts
-> fix, git add, git rebase --continue
[step 2/2] applying E ...      CONFLICT in src/api.ts
-> fix, git add, git rebase --continue
Successfully rebased and updated refs/heads/feature.
```

The Cardinal Rule of Rebase

Never rebase commits that have already been pushed to a shared public repository.

If you must update a remote branch after rebasing your own feature work, protect teammates' unseen commits:

```
git push --force-with-lease # Rejects push if remote has unseen changes
```

Try It: Mini Rebase Drill

```
# Setup
git init demo && cd demo
echo "v1" > a.txt && git add . && git commit -m "init"
git checkout -b feature
echo "feature-line" >> a.txt && git commit -am "feat:␣add␣line"
git checkout main
echo "main-line" >> a.txt && git commit -am "fix:␣hotfix"

# Now rebase feature onto main and resolve the conflict
git checkout feature
git rebase main
# Edit a.txt to keep both lines, then:
git add a.txt && git rebase --continue
git log --graph --oneline --all
```

Section 4

4. Interactive Rebase: Editing Local History

Complete Control Over Commits

Interactive rebase (`git rebase -i`) acts as a local history editor before pushing your updates up to code review.

```
# Open interactive configuration for the last 4 commits  
git rebase -i HEAD~4
```

Command Directives Reference

Directive	Alias	Intended Outcome
<code>pick</code>	<code>p</code>	Keep the commit exactly as it is
<code>reword</code>	<code>r</code>	Keep the code changes but rewrite the log message
<code>edit</code>	<code>e</code>	Stop at this commit to modify files or split it
<code>squash</code>	<code>s</code>	Combine changes into previous commit, merging messages
<code>fixup</code>	<code>f</code>	Combine changes into previous commit, discarding this message
<code>drop</code>	<code>d</code>	Erase this specific commit completely from history

Common History Edits

Workflow 1: Squashing Noise

Turn multiple intermediate “work-in-progress” commits into a clean historical entry.

```
# Todo list modification example
pick abc1234 Add login form structure
f      def5678 Fix layout typo
f      ghi9012 Add input verification rules
```

Workflow 2: Splitting a Monolithic Commit

Break one large commit into small, atomic context pieces.

```
# 1. Mark 'edit' next to the targeted commit in the interactive menu
edit abc1234 Implement complete profile module
```

```
# 2. When Git pauses at the commit, step back one snapshot:
git reset HEAD~1
```

```
# 3. Component Stage and Commit separately
git add component_a.js && git commit -m "Profile_Part_1:_Logic"
git add component_b.js && git commit -m "Profile_Part_2:_Interface"
```

```
# 4. Resume the rebase chain
git rebase --continue
```

Workflow 3: Reordering Commits

Sometimes the *story* of the branch matters: tests first, then implementation. Just rearrange the lines in the todo editor.

# Before	# After (just reorder pick lines)
pick a1 feat: add login impl	pick c3 test: add login tests
pick b2 fix: small typo	pick a1 feat: add login impl
pick c3 test: add login tests	pick b2 fix: small typo

If a reorder produces a conflict, treat it like a normal rebase conflict: resolve, git add, git rebase --continue.

Try It: Clean a Messy Branch

```
# Setup a messy branch
git init demo-irebase && cd demo-irebase
echo "v1" > app.txt && git add . && git commit -m "init"
echo "feat" >> app.txt && git commit -am "Add_featurex"           # bad msg
echo "fix" >> app.txt && git commit -am "WIP"                     # bad msg
echo "fix" >> app.txt && git commit -am "fixup_typo"              # noise

# Now clean it up in one rebase
git rebase -i HEAD~3
#   reword Add featurex          -> "feat: add feature X"
#   reword WIP                  -> "feat: continue feature X"
#   fixup fixup typo
```

Safety Notes

- Interactive rebase **rewrites commit IDs**. Use it freely on private local work.
- Be careful after pushing — coordinate with teammates or use `--force-with-lease`.
- If something goes wrong, `git reflog` is your safety net (Section 6).

Section 5

5. Cherry-Pick: Moving Selected Commits

Targeting Specific Commits

Cherry-pick isolates single commits from other timelines and applies them directly on top of your current branch. The applied commit gets a **new hash** (different parent), so the original is *copied*, not moved.

Before:

A---B---C (main, HEAD)

\

D---E---F (feature)

After 'git cherry-pick E':

A---B---C---E' (main, HEAD)

\

D---E---F (feature, unchanged)

“I need this precise fix, without merging that entire unfinished branch.”

```
# Extract a singular change onto your active branch
git cherry-pick abc1234
```

Practical Application Scenarios

- **Hotfixing:** Pulling a security fix directly from an ongoing testing branch into production.
- **Backporting:** Moving bug fixes from current development releases to legacy support branches.
- **Recovery:** Salvaging valid logic structures from a feature track that was abandoned.

Range and Automation Flags

```
# Single commit
git cherry-pick abc1234

# Inclusive range (oldest^..newest, both ends included)
git cherry-pick abc1234^..def5678

# Replay all commits reachable from master but not from HEAD
git cherry-pick ^HEAD master

# Stage changes without committing - lets you edit before commit
git cherry-pick -n abc1234

# Conflict controls
git cherry-pick --continue
git cherry-pick --abort
git cherry-pick --skip
```

Worked Example: Backporting a Bug Fix

```
# main has moved on; v1.x is a long-lived release branch
$ git log --oneline main -3
e1f2a3b fix: null pointer in parser          <-- we want this on v1.x
a7c8d9e feat: rewrite parser engine
b3c4d5e feat: new config format

$ git checkout v1.x
$ git cherry-pick e1f2a3b
# If parser.c diverged a lot, you may hit conflicts:
# fix the file, then:
$ git add parser.c
$ git cherry-pick --continue
```

Gotcha: Repeated cherry-picks of the same change across long-lived branches can create duplicate-looking commits with different hashes. When you eventually merge, Git usually handles it — but reviewers may be confused. Reference the original commit hash in the message to leave a trail.

Section 6

6. Git Reflog: Recovery and Repair

The Local Safety Net

Reflog tracks every single movement of `HEAD` and branch updates across your local development space.

If a commit was created locally within the past 90 days, Reflog can find and restore it.

Key Mechanics

- Reflog is entirely local; it never travels via `git push` or `git clone`.
- Entries expire after configurable time limits; common defaults are 90 days for reachable entries and 30 days for unreachable ones.

Essential Diagnostics

```
# View complete sequential history of HEAD shifts  
git reflog
```

```
# Sample Output Format:
```

```
# abc1234 HEAD@{0}: reset: moving to HEAD~2
```

```
# def5678 HEAD@{1}: commit: Add database drivers
```

Emergency Recovery Solutions

Scenario A: Accidental Hard Reset Recovery

```
# Recover from an accidental destructive command like:  
git reset --hard HEAD~3  
git reflog  
# Identify pre-reset snapshot state hash (e.g., HEAD@{1})  
git reset --hard HEAD@{1}
```

Scenario B: Restoring an Accidentally Deleted Branch

```
git reflog  
# Locate the last active commit hash belonging to the deleted branch  
git branch feature-restored HEAD@{2}
```

Scenario C: Undoing a Bad Rebase

```
# You just rebased, and the result is wrong.
git reflog
# abc1234 HEAD@{0}: rebase finished: returning to refs/heads/feature
# def5678 HEAD@{1}: rebase: pick "feat: add login"
# ...
# 1a2b3c4 HEAD@{5}: checkout: moving from main to feature <-- pre-
  rebase state
git reset --hard HEAD@{5}
```

Scenario D: Recovering an Amended-Away Commit

```
git commit --amend -m "oops, lost the original message"
# Original commit is unreachable but still in reflog
git reflog show HEAD
# 9f8e7d6 HEAD@{1}: commit: original useful message
git show 9f8e7d6                # inspect
git cherry-pick 9f8e7d6         # bring it back
```

Safety Rule of Thumb

When unsure, create a rescue branch FIRST. Branches are free; lost commits are not.

```
git switch -c rescue HEAD@{1}  # snapshot the suspicious state to a  
    branch  
# now you can experiment without fear
```

Section 7

7. Stash, Worktrees, and Partial Staging

Git Stash: Temporary Workspace Suspension

Save incomplete modifications without generating messy commits when context switching.

```
# Stash tracked modifications  
git stash  
  
# Stash including new, untracked workspace files  
git stash -u  
  
# Describe your stashed changes clearly  
git stash push -m "In-progress_API_refactor"  
  
# Restore the most recent stash (and drop it from the stack)  
git stash pop  
  
# Apply without dropping  
git stash apply stash@{1}  
  
# Review all saved stash layers  
git stash list  
  
# Drop a specific stash entry  
git stash drop stash@{0}
```

Real-World Scenario: The Surprise Hotfix

```
# Working on a half-finished feature when an urgent bug report lands
$ git status
On branch feature/profile
Changes not staged for commit:
  modified:   src/profile.tsx
  modified:   src/api/user.ts

$ git stash push -m "WIP: profile UI redesign"
Saved working directory and index state On feature/profile: WIP: profile
  UI redesign

# Switch, fix, ship
$ git switch main && git switch -c hotfix/login-crash
# ... fix, commit, PR ...

# Back to feature
$ git switch feature/profile
$ git stash pop                                # restores the modifications
```

Git Worktree: Multi-Branch Environments

Check out multiple branches of the same repository simultaneously in separate directories. Unlike `stash`, your files stay where they are — you just open a *second* working directory pointing at a different branch.

Review a PR or execute a hotfix without interrupting or clearing your active workspace.

```
# Mount a separate branch into a sibling directory
git worktree add ../project-hotfix hotfix/api-bug

# See all active worktrees
git worktree list

# Safely tear down directory after finishing the fix
git worktree remove ../project-hotfix
```

Stash vs Worktree: When to Use Which?

Use stash when...	Use worktree when...
Interruption is short (minutes)	Interruption is long (hours/days)
You just need to switch branch and come back	You need to <i>run</i> both branches side-by-side (e.g., diff servers)
Single working tree is enough	You want IDE windows open on two branches

Partial Staging (Patch Mode)

Interactively break down changes within a single modified file into distinct commits — invaluable when one editing session accidentally mixed two unrelated fixes.

```
git add -p # Evaluates code block hunks sequentially
```

Quick Response Map

- **y** : Stage this code hunk.
- **n** : Skip staging this code hunk.
- **s** : Split the current hunk into even smaller evaluation pieces.
- **e** : Manually edit the hunk before staging.
- **q** : Exit immediately; preserve current staging configuration.

Example Flow

```
$ git diff src/utils.ts
@@ -10,3 +10,8 @@ function formatDate(d) { ... }
+function formatTime(t) { ... }          <-- belongs to "feat: time"
@@ -34,1 +39,2 @@
-   return null;
+   return undefined;                    <-- belongs to "fix: nullish"

$ git add -p src/utils.ts
Stage this hunk [y,n,q,a,d,s,e,]? y      # stage formatTime
Stage this hunk [y,n,q,a,d,s,e,]? n      # skip nullish fix
$ git commit -m "feat: add formatTime helper"
$ git add -p src/utils.ts
Stage this hunk [y,n,q,a,d,s,e,]? y
$ git commit -m "fix: return undefined instead of null"
```

Section 8

8. Lazygit: Visual Terminal Workflow

Terminal Interface for Git

Lazygit is a highly responsive keyboard-driven terminal UI that maps complex graph configurations onto clear layouts.

Maintains structural clarity without hiding underlying Git CLI mechanics.

```
# Launch the interface directly in your repository folder  
lazygit
```

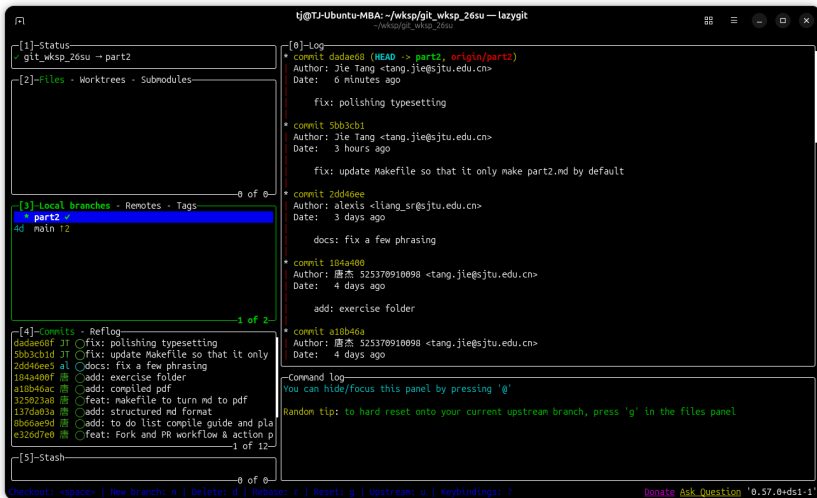


Figure 1: Lazygit Main Interface

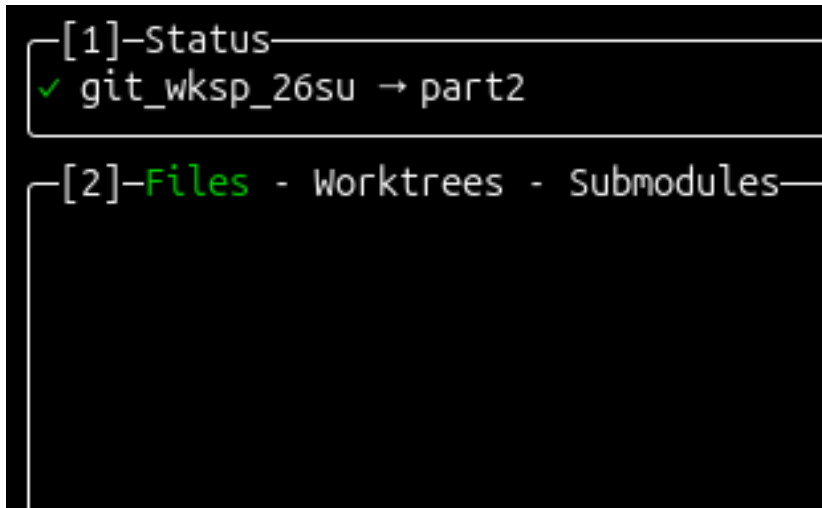
Primary Interface Sections

The interface is split into a **side panel stack** (left) and a **main viewport** (right). Each side panel maps to a numbered shortcut (1–5).

- ① [1] **Status**: current repo & branch, ahead/behind indicators.
- ② [2] **Files**: working tree, staged/unstaged diffs, conflicts.
- ③ [3] **Local branches**: switch, merge, rebase, fast-forward.
- ④ [4] **Commits / Reflog**: graph history & undo timeline.
- ⑤ [5] **Stash**: stash list with apply / pop / drop.

Side Panel

The upper side panel gives an overview of the current repository state, including the active branch, its status, and any pending changes.



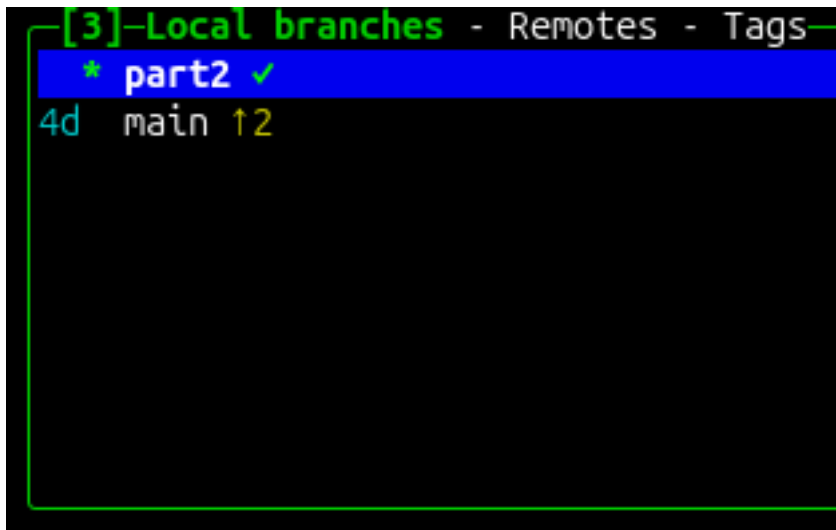
```
[1]-Status-----
✓ git_wksp_26su → part2

[2]-Files - Worktrees - Submodules-----
```

The image shows a terminal window with a dark background. The top section, labeled [1]-Status, shows a green checkmark followed by 'git_wksp_26su → part2'. The bottom section, labeled [2]-Files - Worktrees - Submodules, is currently empty. White lines connect the labels to their respective sections.

Branches Panel

Press 3 to focus this panel.

A screenshot of the Lazygit interface showing the 'Branches' panel. The panel title is '[3]-Local branches - Remotes - Tags' in green. The 'Local branches' section is highlighted in blue and contains the entry '* part2 ✓'. Below it, the 'main' branch is listed with a commit hash '4d' and a yellow arrow pointing to the number '12'.

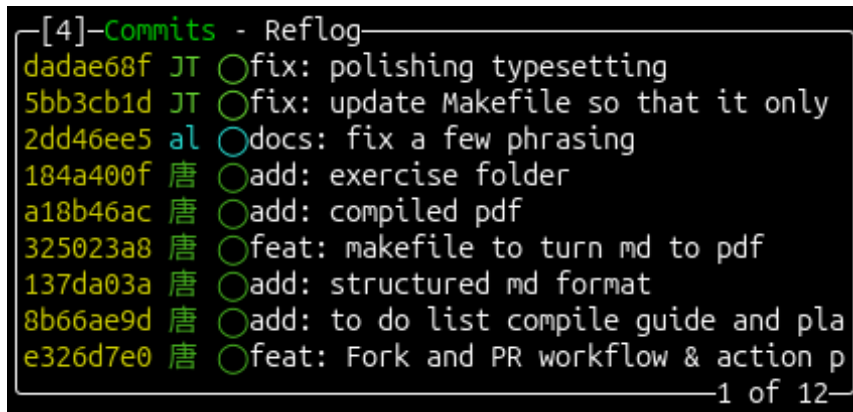
```
[3]-Local branches - Remotes - Tags
* part2 ✓
4d main ↑12
```

Branch Operations Cheat Sheet

Key	Action
space	Checkout the highlighted branch
n	Create a new branch from current
M	Merge into current branch
r	Rebase current branch onto highlighted
d	Delete branch
f	Fast-forward without checkout

Commits Panel & Interactive Rebase

Press 4 to enter the commits panel. This is where lazygit shines for **rewriting history**.



```
[4]-Commits - Reflog
dadae68f JT ○fix: polishing typesetting
5bb3cb1d JT ○fix: update Makefile so that it only
2dd46ee5 al ○docs: fix a few phrasing
184a400f 唐 ○add: exercise folder
a18b46ac 唐 ○add: compiled pdf
325023a8 唐 ○feat: makefile to turn md to pdf
137da03a 唐 ○add: structured md format
8b66ae9d 唐 ○add: to do list compile guide and pla
e326d7e0 唐 ○feat: Fork and PR workflow & action p
1 of 12
```

Figure 4: Commits / Reflog view: history rewriting & cherry-pick from here

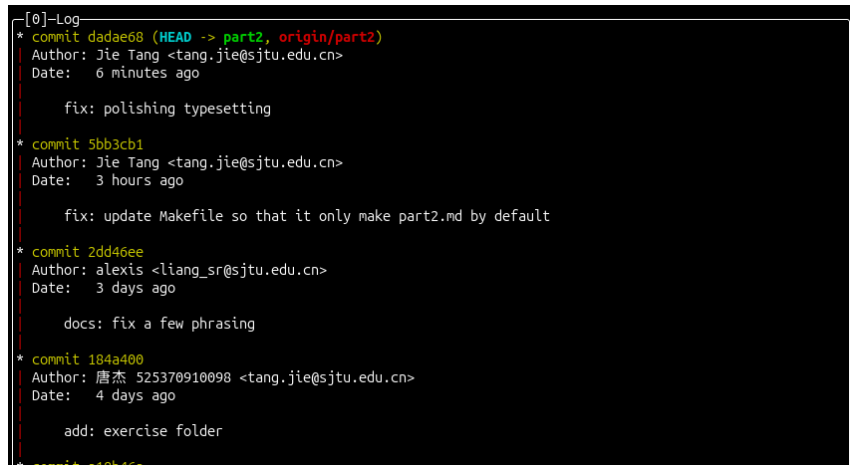
Commits Panel Operations Cheat Sheet

Key	Action
space	Checkout commit (detached HEAD)
e / r	edit / reword this commit (interactive rebase)
s / f	squash / fixup into the commit below
d	drop this commit
p	pick (revert a drop)
c	Cherry-pick selected commit(s)
g	Reset (soft/mixed/hard) to commit
Ctrl+j / Ctrl+k	Move commit down / up (reorder)

*Lazygit performs every action as a real Git command — press @ to toggle the **command log** and watch the underlying `git rebase --interactive ... invocation`.*

Diff / Log Main Viewport

The right pane reflects whatever the active side panel selected — useful for reading commits, diffs, or staging hunks line-by-line.



```
[0]-Log
* commit dadae68 (HEAD -> part2, origin/part2)
| Author: Jie Tang <tang.jie@sjtu.edu.cn>
| Date: 6 minutes ago
|
|     fix: polishing typesetting
|
* commit 5bb3cb1
| Author: Jie Tang <tang.jie@sjtu.edu.cn>
| Date: 3 hours ago
|
|     fix: update Makefile so that it only make part2.md by default
|
* commit 2dd46ee
| Author: alexis <liang_sr@sjtu.edu.cn>
| Date: 3 days ago
|
|     docs: fix a few phrasing
|
* commit 184a400
| Author: 唐杰 525370910098 <tang.jie@sjtu.edu.cn>
| Date: 4 days ago
|
|     add: exercise folder
|
* commit a10b46a
```

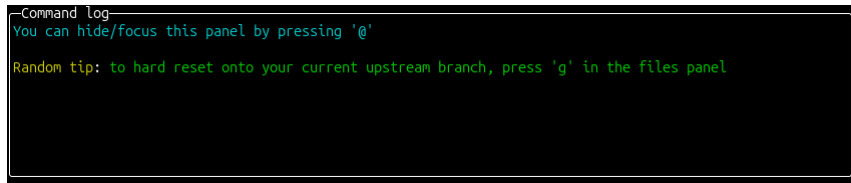
Figure 5: Main viewport showing commit log of the selected branch

When the **Files** panel is focused, `space` here stages an individual file, hunk, or even a single line. Press `Enter` on a modified file to drop into hunk-level review:

[Files panel]		[Diff viewport]
M src/api/auth.ts		@@ -42,7 +42,7 @@
?? notes.todo		- return token;
		+ return signedToken;
		}

Command Log: The Educational Window

Press @ (or look at the bottom panel) to see exactly what Git command lazygit just ran.



```
Command log
You can hide/focus this panel by pressing '@'
Random tip: to hard reset onto your current upstream branch, press 'g' in the files panel
```

Figure 6: Command log: see the actual Git commands lazygit runs

Why this matters: *lazygit is a teacher, not just a tool. Every shortcut maps to a real `git ...` invocation — read along to internalize the CLI.*

Common Operations Cheat Sheet

Workflow	Lazygit Keys	Underlying Git
Stage all & commit	Files $\rightarrow$ a, then c	<code>git add . && git commit</code>
Amend last commit	Files $\rightarrow$ A	<code>git commit --amend --no-edit</code>
Squash commits	Commits $\rightarrow$ s $\times$ N	<code>git rebase -i HEAD~N</code>
Cherry-pick range	Commits $\rightarrow$ Shift+c then Shift+v on target	<code>git cherry-pick A^..B</code>
Resolve conflict	Files $\rightarrow$ Enter on conflicted file	manual edit + <code>git add</code>
Force-with-lease push	Branches $\rightarrow$ p then confirm	<code>git push --force-with-lease</code>
Recover from oops	4 $\rightarrow$ tab to Reflog $\rightarrow$ space	<code>git reset --hard HEAD@{n}</code>

Navigation Basics

- `k` / `j` or arrows : Move up and down within active panels.
- `H` / `L` or `Tab` : Cycle between panels.
- `Space` : Toggle staging for a file / hunk / single line.
- `c` : Open the commit message editor.
- `@` : Toggle the command log window.
- `z` : Undo last operation (`git reset` to a reflog entry).
- `?` : Open the contextual help for the current panel.
- `Esc` or `q` : Step back / exit the interface.

Section 9

9. Remote Collaboration Policies

Keeping Remotes Organized

Fetch vs Pull Optimization

Fetch: Downloads remote updates without modifying local working tracks
`git fetch origin`

Pull: Combines fetch with merge or rebase, depending on configuration
`git pull`

Clean Pull: Replays your local commits cleanly on top of incoming changes
`git pull --rebase`

git pull = git fetch + git merge (default) or git rebase (with --rebase). Fetch first when you want to inspect before integrating.

Tracking Verification

```
# Match local branches with remote destinations explicitly
```

```
git push -u origin feature/auth
```

```
# Inspect tracking health and divergence distances
```

```
git branch -vv
```

Sample Output:

```
* main          abc1234 [origin/main] Fix memory leak  
  feature/auth  def5678 [origin/feature/auth: ahead 1, behind 2] Update  
    tokens
```

- ahead N : you have N commits the remote does not.
- behind N: the remote has N commits you do not.
- Both ahead **and** behind : the branch has *diverged* — a plain `git pull` will produce a merge commit unless you `--rebase`.

Core Safety Controls

- Avoid raw `git push --force`. Always protect your upstream target lines by using `git push --force-with-lease`.
- **Prohibited Action:** Never execute history rewrites or force-pushes on long-lived default branches (e.g., `main`, `master`, `develop`).
- Configure branch **protection rules** on the server (require PRs, status checks, signed commits) so policy is enforced, not just remembered.

Team Policy Checklist

- ☒ Protected `main` / `master` / `dev` (no direct push).
- ☒ All work on feature branches; PRs required.
- ☒ Local noisy commits rebased or squashed *before* review.
- ☒ `--force-with-lease`, never plain `--force` on shared remotes.
- ☒ Ask before rewriting any branch a teammate is using.

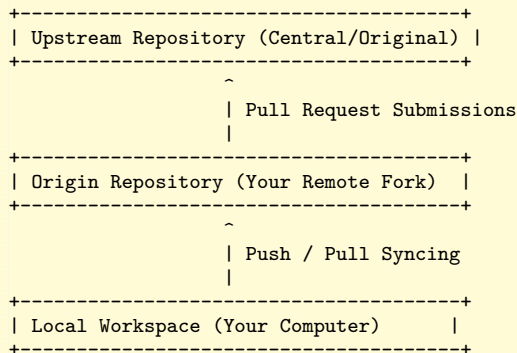
Section 10

10. Fork and Pull Request Workflow

Shared Source Management

A **Fork** is an independent, server-side copy of a repository hosted under your personal namespace.

Repository Ecosystem Map



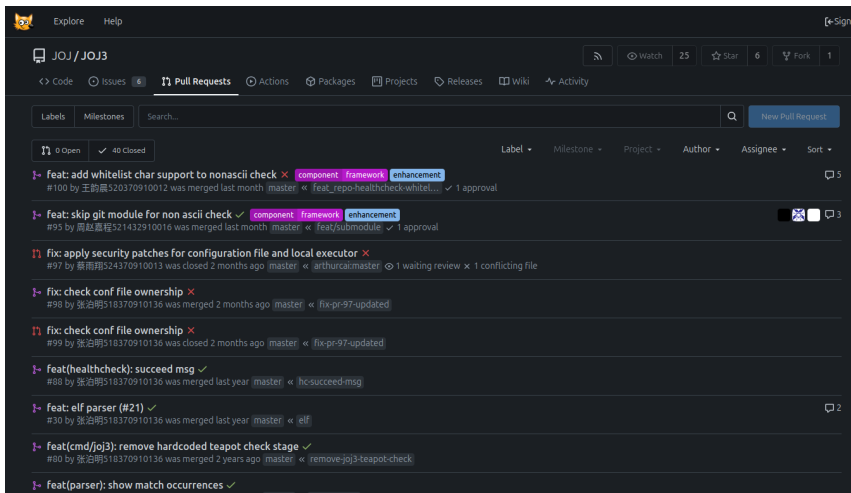


Figure 7: Fork & Pull Request Ecosystem

Step-by-Step Fork Deployment

Step 1: Clone your personal fork

```
git clone https://github.com/YOUR_USERNAME/repository.git
```

Step 2: Establish connection to the original project

```
git remote add upstream https://github.com/ORIGINAL_OWNER/repository.git
```

Step 3: Synchronize with upstream changes before starting new work

```
git checkout main
```

```
git fetch upstream
```

```
git rebase upstream/main
```

```
git push origin main
```

Step 4: Work on your isolated feature track

```
git checkout -b feature/contribution
```

Step 5: Push and open a PR on the upstream repository

```
git push -u origin feature/contribution
```

Then click "Compare & pull request" on GitHub

Keeping the Fork Healthy

```
# Periodically sync main with upstream so your PRs branch from the latest  
base  
git fetch upstream  
git checkout main  
git merge --ff-only upstream/main    # fail loudly if main diverged  
git push origin main
```

Tip: Always branch new feature work from a freshly synced *main*. Diverged forks lead to messy PR diffs.

10. Fork and Pull Request Workflow

What does it mean by students' executor?

I found that I'm wrong. Many stages seem to use the local executor.

 蔡雨翔524370910013 commented 2 months ago

Author

First-time contributor

...

This part is not needed.



张泊明518370910136 reviewed 2 months ago

cmd/joy3/conf/conf.go

@ -186,0 +188,4 @@

```
188 +   if stat, ok := confStat.Sys().(*syscall.Stat_t); ok {  
189 +       uid := int(stat.Uid)  
190 +       currentUid := os.Getuid()  
191 +       if uid != 0 && uid != currentUid {
```

 张泊明518370910136 commented 2 months ago

Owner

...

Will the conf file be owned by root?

 manuel commented 2 months ago

Owner

...

i was wondering the same: we have `student` and `tt`. i think no other user (including `root`) is involved.

 蔡雨翔524370910013 commented 2 months ago

Author

First-time contributor

...

Then uid != 0 is not needed.



 蔡雨翔524370910013 closed this pull request 2 months ago



Some checks are pending

Hide all checks

✗ build / build (pull_request) Failing after 37s

Required

Details

✓ build / trigger-build-image (pull_request) Has been skipped

Details

● build / build (push)

Required

Pull request closed

Please reopen this pull request to perform a merge.

Section 11

11. GitHub Actions: Automation Basics

Declarative Workflow Pipelines

GitHub Actions listen for repository events (e.g., pushes, pull requests) to automatically trigger builds, test suites, and deployments.

Core Concept	Functional Definition
Workflow	Automation blueprint script located within <code>.github/workflows/*.yaml</code>
Event	The explicit trigger condition (e.g., <code>push</code> , <code>pull_request</code>)
Job	A sequence of steps executed on a clean virtual machine runner
Step	Individual tasks that execute console scripts or predefined actions

Standard Continuous Integration Configuration

```
name: Project Verification Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test-suite:
    runs-on: ubuntu-latest
    steps:
      - name: Import Codebase Source
        uses: actions/checkout@v4

      - name: Initialize Runtime Environment
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Execute Standard Testing
        run: |
          npm ci
          npm test
```

The screenshot shows the GitHub Actions interface for a repository named 'JOJ / JOJ3'. The workflow 'feat: add whitelist char support to nonascii check (#100)' is displayed, triggered by a commit '04ae1c8674' pushed by 'manuel' to the 'master' branch. The workflow status is 'Success' and it took '4m27s' to complete.

The workflow steps are as follows:

- sync** (Success, 4m27s)
 - Set up job (42s)
 - Check out repository code (20s)
 - Set up SSH (4s)
 - Set up Git (1s)
 - Git Submodule Update (1m37s)
 - Commit update (28s)


```
1 [master ba8b769] chore: update submodule references [skip ci]
2 1 file changed, 1 insertion(+), 1 deletion(-)
3 remote: . Processing 1 references
4 remote: Processed 1 references in total
5 To ssh://focs.jl.sjtu.edu.cn:2222/JOJ/JOJ3.git
6 04ae1c8..ba8b769 master -> master
```
 - Complete job (49s)

At the bottom, it says 'Powered by Gitea Version: 1.22.0 Page: 33ms Template: 1ms' and 'English | Licenses'.

Figure 9: GitHub Actions Workflow Execution

Production Workflow Best Practices

- **Explicit Pinning:** Use major tags for convenience (e.g., `actions/checkout@v4`) or full commit SHAs when you need stronger supply-chain control.
- **Concurrency Controls:** Cancel older runs automatically when a developer pushes new updates to an active pull request:

```
concurrency:  
  group: ${{ github.workflow }}-${{ github.ref }}  
  cancel-in-progress: true
```

Section 12

12. Capstone Exercise: Clean Up a Messy Team Repository

The Scenario

Your team is preparing a release. The development branches are cluttered with repetitive, unverified commits, someone accidentally ran an incorrect rebase, and a vital hotfix is stuck on a abandoned branch. Your objective is to clean up this repository structure.

Practical Execution Tasks

- 1 Run the `git log --graph --oneline --decorate --all` command to map out all branch locations.
- 2 Locate and recover a lost commit using `git reflog`.
- 3 Rebase your target feature branch onto the latest `main` branch state.
- 4 Manually resolve the resulting rebase conflicts, then resume using `--continue`.
- 5 Run an interactive rebase (`git rebase -i`) to squash small commits and clean up your log messages.
- 6 Use `git cherry-pick` to bring over the isolated hotfix from the abandoned branch.
- 7 Merge the polished feature branch into `main` using your team's integration strategy.
- 8 Verify the final graph state and staging layout inside `lazygit`.
- 9 Safely push the completed history up to the remote server using `--force-with-lease`.

Thank you!